TRACE

Wallet intelligence that learns, explains, and earns trust.

PRODUCT REQUIREMENTS DOCUMENT

A production-minded specification for a privacy-conscious Sui wallet intelligence platform powered by Tatum and Walrus Memory.

PRODUCT CATEGORY	CORE STACK
Wallet analytics · behavioural anomaly detection · explainable AI	Web app · API backend · Tatum Sui RPC · Walrus Memory
DESIGN DIRECTION	**QUALITY BAR**
Premium light UI · editorial typography · restrained green · purposeful motion	Typed contracts · observable services · tested security boundaries · reproducible demos

Document intent

Define the product, user experience, architecture, data model, privacy and security posture, integration contracts, quality gates, and measurable acceptance criteria. This document intentionally contains no delivery timeline.

Ambition note: TRACE is designed to be credible as an extensible product, not to claim that an MVP is already enterprise-certified, audited, or guaranteed to attract an acquisition offer.

Version 1.0 · Prepared 27 October 2026

Contents

Navigate by product concern. Each section is written to support implementation, review, and judging.

1. Executive summary
2. Problem, vision, and product principles
3. Target users and jobs to be done
4. Scope and non-goals
5. Product experience and key journeys
6. Information architecture and screen requirements
7. UI system, visual references, and motion
8. Functional requirements
9. Learning loop and memory design
10. Blockchain and Tatum integration
11. Privacy, security, and threat model
12. System architecture
13. Data model and API contract
14. AI / anomaly detection methodology
15. Reliability, observability, and operations
16. Testing and quality gates
17. Deployment and configuration
18. Accessibility and performance
19. Metrics and acceptance criteria
20. Demo narrative and judging evidence
21. Risks and mitigations
22. Repository structure and engineering standards
23. Reference links

1. Executive summary

TRACE is a wallet intelligence product for Sui that helps users understand activity in a wallet, distinguish routine behaviour from unusual events, and receive explanations grounded in observable on-chain evidence.

The core differentiator is not a chat interface. TRACE maintains a user-controlled, persistent memory of preferences, known counterparties, prior alerts, feedback, and verified outcomes. It retrieves relevant memories in later sessions and uses them to contextualise new activity. The app must demonstrate a measurable before/after difference in a controlled scenario.

Product promise

- Understand what changed in a Sui wallet and why it may matter.
- See the transaction facts supporting every material claim.
- Learn from confirmed or dismissed alerts without silently treating feedback as ground truth.
- Keep private preferences and notes separate from public blockchain facts.
- Provide a clear path from wallet overview to evidence, alert explanation, memory provenance, and user correction.

Product positioning

TRACE is an explainable wallet-risk and behavioural intelligence layer, not a custody product, trading bot, financial adviser, sanctions screening service, or guarantee that a wallet is safe. It is designed first for read-only analysis and transparent explanations.

Hackathon integration thesis

Component	Required product role	Evidence to show
Tatum Sui RPC / Data	Retrieve wallet-related on-chain facts, balances and transaction/object details supported by the chosen endpoint.	Request logs with secrets redacted; normalized transaction details; source and timestamp.
Walrus Memory	Persist and recall user-approved profile preferences, known-entity labels, alert feedback and investigation summaries.	Successful Mainnet memory jobs; at least 10 genuine memory records; a later-session recall changing an explanation.
AI / analysis layer	Generate a bounded explanation over validated facts and recalled memory.	Cited facts, structured output, confidence/uncertainty, and deterministic fallback.
Frontend	Make data freshness, risk reasoning, memory use, and privacy boundaries legible.	A polished, navigable demo with meaningful loading, empty, error, and success states.

2. Problem, vision, and product principles

Problem

Wallet interfaces often expose raw activity without translating it into a user's context. Generic alerts may repeat, miss preferences, or provide little evidence. A separate AI chat can summarise transactions but forgets why an earlier alert was dismissed or which counterparties the user explicitly marked as familiar. This creates noisy experiences and weak trust.

Vision

A user should be able to return to TRACE after weeks away and find an assistant that understands their declared preferences, remembers prior investigations, and can explain how a new observation compares with earlier patterns—without pretending that public-chain data is private or that correlation proves fraud.

Product principles

- **Evidence before confidence.** Every on-chain claim links to the underlying transaction/object or an explorer link when available.
- **Memory must change behaviour.** Memory retrieval should affect ranking, context, or explanation in a testable way.
- **Privacy by minimisation.** Store only what is needed; let users inspect, correct, and remove app-managed memory where supported.
- **Read-only by default.** Never request seed phrases or private wallet signing keys. Do not initiate transfers in the MVP.
- **Explain uncertainty.** “Unusual” is not the same as “malicious.” Separate facts, inferences, and unknowns.
- **Resilient by design.** If Tatum or Walrus Memory is unavailable, degrade gracefully and disclose the limitation.
- **Accessible and calm.** High contrast, semantic states, keyboard support, restrained motion, and no alarmist colour usage.
- **Real over theatrical.** No fake live data, fabricated transaction histories, simulated integrations presented as real, or made-up security claims.

Product narrative

TRACE starts with a wallet snapshot, builds a baseline from available history, surfaces a notable deviation, explains it with evidence, asks for user feedback where appropriate, and carries the validated context into the next session. The product becomes more useful because it remembers relevant context—not because it claims to train a foundation model on every interaction.

3. Target users and jobs to be done

Persona	Need	TRACE job	Guardrail
Individual Sui user	Understand wallet activity without reading every transaction.	Summarise notable changes and explain transactions in plain language.	Never imply a wallet is safe based on a limited history.
Power user / trader	Spot deviations and keep track of recurring counterparties.	Compare current activity with user-approved patterns and previous investigations.	No investment recommendation or price-performance promise.
Developer / integrator	Embed contextual wallet intelligence in another product.	Expose documented read-only APIs and structured evidence objects.	Scope API keys, rate limits, and per-user data isolation.
Security-conscious user	Know what is public, what is remembered, and how to correct it.	Provide memory review, provenance, consent, export/delete flows where feasible.	Do not promise blockchain erasure or absolute anonymity.
Analyst / support team (future)	Reproduce why an alert was generated.	View evidence references and versioned rule/model decisions.	No access to private user memories by default.

Jobs to be done

- When I connect a wallet address, I want to understand its recent activity without manually opening every transaction.
- When an alert appears, I want to know exactly which facts caused it and what is still uncertain.

- When I label a counterparty or correct an alert, I want future explanations to use that context.
- When I return in a new session, I want TRACE to recall relevant preferences without inventing memories.
- When I inspect a memory, I want to know when it was created, what source it came from, and whether it influenced a result.

4. Scope and non-goals

MVP — must ship

- Wallet address input with validation and read-only analysis.
- Tatum-backed Sui data retrieval through a server-side integration module.
- Normalized wallet activity view with clear freshness and partial-data indicators.
- Rule-based behavioural baseline and explainable anomaly candidates.
- Walrus Memory integration using the official SDK and Mainnet configuration.
- At least 10 successful, meaningful Mainnet memory writes; job completion must be awaited and recorded.
- Recall relevant memories during a later analysis and show how they affected the result.
- Alert detail view with evidence, rationale, uncertainty, and user feedback.
- Memory centre to view app-managed memory summaries and their provenance.
- Authentication or a deliberate demo-mode boundary; tenant isolation; rate limiting.
- Public repository, setup instructions, tests, demo, and technical write-up.

Should ship if reliable

- Watchlist for multiple wallet addresses.
- Saved investigations and user-defined labels.
- Export of an investigation report with evidence links.
- A clearly marked read-only public demo wallet.

Explicit non-goals for the initial release

- Custody, seed phrase handling, transaction signing, automatic transfers, or autonomous asset movement.
- Guaranteeing detection of all scams, sanctions exposure, exploits, or illicit activity.
- Private-chain analytics or claiming that public Sui transactions are confidential.
- A token launch, speculative trading signal, copy trading, or profit claims.
- Training a custom large language model or building a general-purpose autonomous agent.
- Multi-chain support before the Sui experience is reliable.
- Enterprise certifications, third-party security audit, or compliance status not actually obtained.

Scope rule: a feature is not complete because a screen exists. It is complete when data provenance, error states, authorization, tests, and the demo path work end to end.

5. Product experience and key journeys

Journey A — First wallet analysis

- User lands on a concise value proposition and enters a Sui address. The UI explains that the address and on-chain activity are public.
- Backend validates the address format and applies rate limits before calling Tatum.
- TRACE fetches supported balance and activity data, normalizes it, and records source, fetch time, and pagination status.
- Analysis engine establishes a baseline only from the available observation window and marks thin-history cases.
- User sees a wallet overview, notable activity, and evidence-linked explanations.
- TRACE offers to remember explicitly selected preferences or labels. It does not silently store sensitive free text.

Journey B — A notable transaction

- A transaction appears that differs from the observed baseline or a user-defined rule.
- The system creates an alert candidate with deterministic reasons, evidence IDs, severity, and uncertainty.
- The explanation layer receives only the minimum required transaction facts plus relevant recalled memories.
- The UI distinguishes on-chain facts from inferred risk signals and shows source links.
- User marks the alert as expected, needs review, or incorrect, with an optional reason.
- Feedback is stored as a versioned memory only after confirmation and is never treated as definitive proof of safety.

Journey C — Return in a later session

- User opens the wallet again or starts a new investigation.
- TRACE recalls only memories relevant to the wallet and task, using an explicit namespace and tenant scope.
- The UI identifies relevant remembered context, e.g. “You marked this address as your exchange account on [date].”
- The explanation reflects the context and visibly notes whether memory changed the result.
- User can inspect, correct, or request removal of the app-managed memory.

Journey D — Provider failure

If Tatum is unavailable, TRACE displays the last successful fetch timestamp, labels the result stale, and disables claims that require fresh data. If Walrus Memory is unavailable, the app may continue a stateless analysis if safe, but it must disclose that persistent context was not retrieved or saved. Failed memory jobs must not be counted toward the 10-write requirement.

6. Information architecture and screen requirements

Screen	Primary content	Key interactions / states
Landing / connect	Clear proposition, supported network, privacy note, address input.	Validation, example address, loading, unsupported network message.
Wallet overview	Balance summary where supported, activity feed, freshness, watchlist state.	Time filter, pagination, refresh, empty history, partial data.
Activity detail	Transaction/object details, timestamp, involved addresses, explorer link, evidence IDs.	Copy address/hash, open explorer, report missing/incorrect parsing.

Screen	Primary content	Key interactions / states
Risk / insight feed	Prioritised anomaly candidates with reason chips and severity.	Filter, dismiss with reason, mark expected, open evidence.
Investigation detail	Question, observed facts, rule/model version, reasoning, caveats, sources.	Copy/share report, inspect source, submit correction.
Memory centre	Memory summary, type, creation time, source, last used, namespace.	Inspect, edit/delete app-managed record if supported, export, consent controls.
Settings & privacy	Data handling summary, retention controls, connected services, session settings.	Delete account data, revoke sessions, manage analytics consent.
System status / diagnostics	Provider health, last successful Tatum read, memory-write job status.	Retry, view request ID, copy redacted diagnostic details.

Global interface states

- Loading skeletons that preserve layout; no indefinite spinner without a timeout.
- Empty state explains why there is no data and what action is possible.
- Stale state shows the timestamp and disables “live” claims.
- Error state offers a safe retry and a non-secret request ID.
- Partial state explains which provider or pages failed.
- Success state confirms what actually succeeded, not merely that a request was sent.
- Destructive actions require clear confirmation and explain what cannot be erased from a public blockchain.

7. UI system, visual references, and motion

Visual target: a premium, confident financial intelligence product. Use a bright canvas, near-black green-tinted text, restrained forest-green accents, warm off-white surfaces, hairline borders, disciplined typography, and carefully designed data visualisation. Do not use purple, dark-first UI, generic AI gradients, glowing blobs, or decorative dashboards full of meaningless charts.

Reference products and what to learn

Reference	Study	Do not copy blindly
Stripe	Editorial hierarchy, documentation clarity, high-quality product language, precise interaction feedback.	Brand assets, proprietary layouts, or exact copy.
Linear	Keyboard-first interaction, purposeful transitions, compact density, consistency.	Dark mode as the default visual direction.
Ramp	Financial-product clarity, information hierarchy, confident but restrained use of colour.	Claims or features not present in TRACE.
Coinbase	Trust cues, asset information, approachable explanations of complex financial data.	Making risk indicators look like investment advice.
Vercel	Developer-grade system status, polished empty/loading states, clear deployment and diagnostics language.	Overly technical language for ordinary users.

Design tokens

Token	Specification
Canvas	#FAFAF7 / warm neutral; use pure white for high-focus surfaces where useful.
Primary text	#17231F; secondary text #5F6C66. Meet WCAG contrast requirements.
Primary action	Forest green #176B50; use sparingly for primary actions and positive states.
Borders	#DCE4DE; thin, consistent, and not used as the only state indicator.
Risk states	Neutral by default; amber for needs-review; red only for high-severity, evidence-backed warnings. Always pair colour with text/icon.
Typography	Use a high-quality sans family (e.g. Geist, Inter, or system fallback) with tabular numerals for balances, dates, and metrics.
Radius	Small-to-medium radii; avoid excessive pill cards. Establish a consistent 8/12/16/24 spacing scale.
Charts	Minimal axes, clear units, accessible labels, source/freshness context, and no fabricated data.

Motion system

- Use Framer Motion for route transitions, expandable evidence panels, alert arrival, memory-use indicators, and subtle chart reveals.
- Motion must clarify state changes, not hide latency or simulate data processing that did not occur.
- Default transitions should be brief and restrained; stagger only small groups of related elements.
- Provide reduced-motion behaviour using the platform preference. Avoid parallax and large, continuous movement.
- Every animation must retain keyboard focus, avoid layout shifts, and preserve readable content.
- Use SVG for small, purposeful diagrams (evidence paths, data lineage) rather than decorative moving backgrounds.

Component library strategy

Use a maintained accessible component foundation such as Radix UI primitives with a consistent Tailwind token layer, or shadcn/ui as a composition starting point. Keep business logic out of visual components. Build a small internal design system with documented variants, not a pile of one-off styles. Verify dependency licences and accessibility behaviour before adoption.

8. Functional requirements

ID	Requirement
FR-01 Address validation	Validate the supported Sui address format and reject malformed input before any provider call.
FR-02 Read-only analysis	The MVP never requests a seed phrase, private wallet key, or transaction-signing permission.
FR-03 Tatum data	Fetch supported wallet-related facts through server-side Tatum integration; record source, network, fetched_at, and provider request ID.
FR-04 Normalization	Normalize provider payloads into internal schemas with explicit unknown/null fields; do not silently coerce missing values to zero.
FR-05 Pagination	Respect provider pagination/limits; expose incomplete-history status when the observation window is partial.

ID	Requirement
FR-06 Baseline	Compute interpretable baselines from available history and label low-sample results.
FR-07 Alert generation	Create explainable candidates with evidence references, rule version, severity, and uncertainty.
FR-08 Memory write	Write user-approved memory through Walrus Memory SDK and await the job completion before reporting success.
FR-09 Memory recall	Retrieve only relevant memories in the correct user/app namespace and attach memory IDs to the analysis trace.
FR-10 Feedback	Store user feedback with timestamp and provenance; distinguish user assertion from independently verified fact.
FR-11 Memory controls	Allow users to inspect memory summaries and manage app-controlled records where supported by the SDK/product.
FR-12 Observability	Provide structured logs and request IDs without secrets, raw sensitive memory text, or unnecessary full addresses.
FR-13 Degraded mode	Disclose provider failure/staleness and avoid presenting stale data as current.
FR-14 Demo integrity	Clearly label sample/demo data. Never mix demo data into a live wallet's analysis.
FR-15 Documentation	Document local setup, env variables, threat model, supported RPC methods, memory namespace strategy, and limitations.

9. Learning loop and memory design

Memory categories

Category	Example	Handling rule
User preference	“Explain risk in concise language.”	Explicit opt-in; user can inspect and change.
Counterparty label	“Address X is my exchange deposit address.”	User-declared, timestamped, never independently verified by default.
Alert feedback	“This recurring transfer is expected.”	Store as feedback, not proof that future transfers are safe.
Investigation summary	“Transaction A differed due to a new recipient and unusual amount.”	Include source references and analysis version.
Verified outcome	A prior alert was later confirmed by a trusted source.	Store verification source and confidence; separate from model inference.
Sensitive information	Seed phrase, private key, passwords, recovery codes.	Never collect or store.

Memory record envelope

```
memory_type, tenant_id, wallet_ref, created_at, updated_at, source_type, source_ref, user_confirmed,
confidence, expires_at, schema_version
```

This envelope is an application-level conceptual model; it must be mapped to the official Walrus Memory SDK capabilities rather than assumed to be a native schema. Keep raw memory text minimal and avoid placing full personal information in memory when a compact reference or pseudonymous label is sufficient.

Write and recall rules

- Use an explicit namespace per environment and a documented isolation strategy per tenant/user. Confirm actual isolation guarantees against current SDK documentation before production use.
- Write only after an intentional event: user-confirmed preference, labelled counterparty, meaningful alert feedback, or completed investigation summary.
- Wait for the memory job to finish and persist its job status; accepted is not equivalent to successfully stored.
- Record a local non-secret audit event containing memory type, job ID, completion status, and timestamp—not the private key or unnecessary raw text.
- Recall before analysis only when relevant; limit returned memories and attach provenance to the result.
- Resolve conflicting memories using timestamps and explicit user confirmation; never let old memory silently override fresh chain evidence.
- If recall fails, proceed only with a clearly labelled stateless analysis or return a safe error.
- Create a repeatable demo that stores at least 10 meaningful Mainnet memories and verifies later recall; do not fabricate writes or count failed jobs.

Learning-loop acceptance test

Given a wallet with a recurring transfer and a user-confirmed label, when the same address is analyzed in a new session, TRACE must recall the label, show its provenance, and change the alert explanation or priority where the label is relevant. The test must also show the output without memory and prove the difference. If the memory does not affect a user-visible decision, the learning-loop requirement is not met.

10. Blockchain and Tatum integration

Integration boundary

All Tatum calls run on the server. The browser never receives the Tatum API key. A typed provider adapter isolates vendor-specific request formats from product logic. Use Tatum's currently supported Sui RPC interface and verify the exact methods/response schema in the live documentation during implementation.

Candidate read capabilities

Need	Possible Sui capability	Implementation note
Network identification	Chain identifier / network health method	Verify network is Mainnet or Testnet as configured.
Wallet balances	suix_getAllBalances or gRPC StateService/ListBalances	Use supported Tatum endpoint; normalize coin type, raw amount, decimals, and display amount separately.
Recent transactions	suix_queryTransactionBlocks or gRPC LedgerService/ListTransactions	Confirm address filter, pagination, ordering, and provider limits.
Transaction detail	sui_getTransactionBlock or gRPC LedgerService/GetTransaction	Request only fields needed; preserve digest, timestamp/checkpoint, status, and evidence refs.
Owned objects / token context	suix_getOwnedObjects or gRPC StateService/ListOwnedObjects	Handle object types and metadata defensively; do not assume every object is a token.
Event context	suix_queryEvents / supported event APIs	Only add if needed by a concrete feature and documented response semantics.

Sui's documentation notes that JSON-RPC was disabled on Sui Foundation Mainnet full nodes in 2026. Tatum documents a Sui gRPC endpoint; prefer the currently supported Tatum interface and avoid hard-coding an obsolete

provider path. Verify the endpoint and API key header against current Tatum docs.

Data provenance contract

- Every fact has chain/network, source method, fetched timestamp, and stable transaction/object identifier when available.
- Distinguish on-chain timestamp from time observed by TRACE.
- Handle missing timestamps and failed transactions explicitly.
- Use decimal-safe arithmetic for token quantities; never use floating-point arithmetic for exact raw amounts.
- Do not infer fiat value unless a trustworthy price source is explicitly integrated and labelled.
- Cache read-only results with bounded TTL and stale markers; never conceal cache age.
- Use retry with exponential backoff and jitter for transient provider failures; cap retries and respect rate limits.

Secrets and environments

Use distinct Tatum Testnet and Mainnet keys in local environment configuration. Never commit keys. Never send keys to the browser, logs, screenshots, AI prompts, or client-side telemetry. Rotate any key that may have been exposed. Restrict allowed use/permissions where the provider supports it.

11. Privacy, security, and threat model

Security posture: privacy-conscious and production-minded, but not a claim of formal audit or certification. Public blockchain data is public. Walrus Memory's documented SDK/relayer handles encryption, upload, and indexing; the implementation must verify the actual SDK configuration and access model and must not invent stronger guarantees than the protocol provides.

Data classification

Class	Examples	Controls
Public chain data	Transaction digests, public addresses, publicly visible balances/activity.	Treat as public; minimise duplication; display network/source and avoid claims of confidentiality.
Private app data	Preferences, user notes, alert feedback, investigation annotations.	Access control, minimal retention, tenant isolation, encrypted transport, explicit memory controls.
Secrets	Tatum API keys, Walrus delegate private key, auth signing secrets.	Server-side secret manager/environment; no client exposure; redact logs; rotation procedure.
Operational metadata	Request IDs, latency, provider status, job state.	Minimise; pseudonymise identifiers; retention policy; no raw private content in logs.

Threat model

Threat	Mitigation / required test
API key theft	Server-only keys, secret manager, redaction tests, key rotation guide.
Cross-user data leakage	Authenticated tenant context, namespace strategy, authorization on every object, negative isolation tests.
Prompt injection in user labels / metadata	Treat chain metadata and user text as untrusted data; delimit data from instructions; never allow memory content to authorize actions.
False positive risk label	Evidence-backed reasons, confidence/uncertainty, feedback path, no categorical “scam” claim without strong corroboration.
Malicious or malformed RPC payload	Schema validation, response-size limits, safe parsing, no unsafe HTML rendering.
Memory replay / stale context	Timestamped records, source and version metadata, conflict handling, fresh chain data takes precedence.
Private data over-collection	Allowlist memory types, redact sensitive patterns, no seed phrase/password storage, retention controls.
Abuse / denial of service	Rate limits, request caps, timeouts, bounded concurrency, provider-cost monitoring.
XSS / CSRF / session abuse	Framework security defaults, output encoding, secure cookies, CSRF controls as appropriate, CSP, dependency scanning.
Unsafe transaction signing	Out of scope; no seed phrases, no signing keys, no transaction execution in the initial release.

Privacy promises we may make

- “TRACE is read-only in the initial release.”
- “Your Tatum and Walrus credentials stay on the server and are not included in browser bundles.”
- “Public chain data remains public; TRACE does not make blockchain activity private.”
- “Private app memory is handled through the documented Walrus Memory flow and application access controls.”
- “We show what is remembered and provide available controls for managing app-managed memory.”

Promises we must not make

- “Your wallet activity is anonymous/private because it is stored on Walrus.”
- “All scams are detected” or “this wallet is guaranteed safe.”
- “Deleting an app memory erases public blockchain records.”
- “The product is audited, compliant, bank-grade, or enterprise-certified” unless independently established.

12. System architecture

Recommended implementation is a modular monolith with clear boundaries rather than premature microservices. It keeps the initial codebase coherent while allowing future extraction of ingestion, analysis, or integrations if scale demands it.

WEB CLIENT
Next.js + TypeScript · accessible component system · server actions/API client · charts · motion

APPLICATION API
FastAPI · Pydantic schemas · auth/session middleware · rate limits · request IDs · structured errors
DOMAIN MODULES
Wallet service · Tatum adapter · normalization · baseline/rules engine · explanation service · memory service · audit trail
EXTERNAL SYSTEMS
Tatum Sui RPC / Data APIs · Walrus Memory Python SDK or documented SDK adapter · model provider (optional, server-side)
PERSISTENCE / OPERATIONS
PostgreSQL for app metadata · Redis only if needed for rate limit/cache/jobs · structured logs · error monitoring · metrics

Stack recommendation

Layer	Preferred choice	Rationale
Frontend	Next.js + TypeScript	Strong ecosystem for product UI, routing, server rendering, and polished interactions.
UI	Tailwind CSS + Radix primitives / shadcn/ui	Composable, accessible foundation with full control over visual direction.
Motion	Framer Motion	Purposeful transitions and reduced-motion support.
Backend	FastAPI + Pydantic	Typed API contracts, async support, clear modular architecture.
Walrus Memory	Official Python SDK if supported for required flows; otherwise a small, tested adapter to the official documented SDK/API.	Avoid custom cryptography or undocumented protocol calls. Validate SDK parity and Mainnet write flow early.
Database	PostgreSQL	Durable users, investigations, evidence refs, job status, and audit metadata.
Cache / jobs	Start with bounded in-process tasks only for local/demo if safe; add Redis/worker only when required.	Avoid unnecessary infrastructure; persist job status and retry semantics.
Testing	Pytest, HTTPX, TypeScript type-check, Playwright	Unit, contract, integration, browser, and end-to-end coverage.
Deployment	Frontend and API deployed separately with server-side secrets	Independent scaling and clearer failure isolation.

Architecture decision gate: confirm the current Walrus Memory Python SDK supports the exact Mainnet write/recall functions needed. If not, use the official TypeScript SDK behind a narrow integration service rather than implementing signing, encryption, embeddings, or storage internals from scratch.

13. Data model and API contract

Core entities

Entity	Important fields	Notes
User	id, auth_subject, created_at, privacy_settings	Never store wallet secrets. Auth implementation must be explicit.
WalletWatch	id, user_id, chain, address, label, created_at	Address is public data but user-to-address association can be sensitive.
ChainObservation	id, wallet_watch_id, source, method, fetched_at, cursor, completeness, payload_ref	Avoid unnecessary raw payload retention; store normalized facts.
TransactionFact	digest, network, timestamp, status, sender/recipient if available, token/object references	Preserve unknown values and source provenance.
Alert	id, wallet_watch_id, severity, reasons[], evidence_refs[], rule_version, created_at, state	Alert is a hypothesis or signal, not a verdict.
Feedback	id, alert_id, user_id, label, reason, created_at, verified_status	User feedback and verified outcome must remain distinct.
MemoryAudit	id, user_id, namespace_ref, memory_type, job_id, status, created_at, last_recalled_at	Do not store delegate private key or unnecessary raw memory text.
Investigation	id, user_id, question, summary, evidence_refs[], analysis_version, created_at	Keep source IDs and versioning for reproducibility.
ProviderHealth	provider, status, last_success_at, latency_ms, error_code	No credentials or raw sensitive payloads.

API surface — illustrative, versioned contract

Method / route	Purpose	Security and response notes
POST /api/v1/wallets/analyze	Validate address, fetch supported data, return normalized snapshot and analysis.	Rate limited; user context required outside public demo; request ID included.
GET /api/v1/wallets/{id}/activity	Paginated normalized activity.	Authorization check; explicit cursor and completeness.
GET /api/v1/alerts	List alert candidates.	Tenant scoped; filters bounded.
GET /api/v1/alerts/{id}	Return evidence and explanation.	Every fact linked to evidence/source.
POST /api/v1/alerts/{id}/feedback	Record user response.	Idempotency key; validate enumerated feedback types.
GET /api/v1/memories	List app-managed memory audit summaries.	Do not expose private delegate key or unrelated namespaces.
POST /api/v1/memories	Request an approved memory write.	Allowlisted memory types; job status tracked.
GET /api/v1/memories/jobs/{job_id}	Check completion of asynchronous memory write.	Accepted/pending/succeeded/failed states explicit.
GET /api/v1/health	Liveness/readiness summary.	Public response must not disclose secrets or internal topology.

API standards

- Use Pydantic request and response models; reject unknown dangerous fields where appropriate.

- Version public APIs and document schemas with OpenAPI.
- Use consistent errors: code, safe message, request_id, retryable boolean.
- Apply timeouts, bounded payloads, pagination limits, and idempotency to mutating endpoints.
- Never return upstream credentials or unrestricted provider error bodies.
- Do not expose a general-purpose endpoint that lets clients proxy arbitrary Tatum RPC calls.

14. AI / anomaly detection methodology

Prefer a transparent hybrid system

The MVP should start with deterministic, testable rules and simple statistical baselines. A language model may explain the findings, but it must not invent transaction facts or be the sole source of risk scoring. This is more defensible than a black-box “AI risk score” with no labelled training data.

Candidate signals

- New or previously unseen counterparty relative to the observed window.
- Amount materially outside a user's own observed range, where comparable units are available.
- Unusual transaction frequency or burst relative to a sufficient baseline.
- Unexpected asset/object type or transaction pattern, if the required fields are reliably available.
- A mismatch between a remembered user label and fresh chain facts, flagged as a review prompt rather than a definitive threat.

Scoring model

Represent each signal as a structured feature with observed value, baseline, source, rule ID, and contribution. Combine signals using a versioned rule configuration. Keep severity distinct from confidence: severity describes potential impact; confidence describes evidence quality. For low-history wallets, return “insufficient history” rather than an overconfident score.

LLM explanation contract

- Input only validated normalized facts, selected evidence references, relevant memories, and explicit instructions.
- Output structured JSON validated against a schema: summary, observed_facts, inferences, uncertainty, next_steps, citations, memory_used.
- Every factual statement about a transaction must map to an evidence reference.
- If a claim cannot be supported by supplied evidence, the model must say it cannot determine the answer.
- Never allow the model to execute transactions or modify wallet data.
- Treat wallet metadata, token names, user labels, and recalled text as untrusted content—not as system instructions.
- Persist model/provider version and prompt template version for reproducibility, excluding sensitive prompts from general logs.

No false precision

Do not display “92% chance of fraud” unless there is a validated calibration dataset and a defensible statistical basis. Prefer interpretable labels such as “unusual relative to this wallet's observed history” with a confidence note and evidence. Do not train or evaluate on synthetic labels and then present results as real-world accuracy.

15. Reliability, observability, and operations

Operational requirements

- Structured JSON logs with timestamp, level, service, route, request_id, provider, latency, and safe error code.
- Metrics for Tatum request success/error/latency, memory write completion/failure, recall latency, analysis duration, and rate-limit events.
- Correlation IDs across frontend request, backend operation, Tatum call, analysis, and Walrus Memory job.
- Health endpoints distinguish process liveness from dependency readiness.
- Timeouts and bounded retries for external calls; no infinite retry loops.
- Circuit-breaker or short cooldown behaviour for repeated provider failures.
- Cost controls: cap history depth, result count, model tokens, request frequency, and concurrent analyses.
- Audit events for memory writes, deletes/retention actions, and feedback updates.
- Alert on failed memory writes and provider degradation; never count accepted-but-incomplete writes as successful.

Operational runbooks

- Tatum key invalid/expired: confirm server environment, rotate key, verify header, and check rate limits.
- Walrus Memory write failure: inspect job state and safe error, retry only according to SDK semantics, avoid duplicate writes where possible.
- Stale chain data: show last-success timestamp and provider status; do not label it live.
- Unexpected cost spike: throttle requests, reduce history depth, inspect per-route metrics, and disable expensive analysis safely.
- Possible credential leak: rotate the credential, inspect repository history/logs, remove from current source, and treat history cleanup as a separate task.

16. Testing and quality gates

Test layer	Required coverage	Pass condition
Unit	Address validation, amount parsing, normalization, baseline rules, severity/confidence separation.	Deterministic tests; edge cases and malformed inputs covered.
Contract	Tatum response fixtures and Walrus SDK adapter contract.	Schema drift or missing fields fail safely and visibly.
Integration	Real Tatum Testnet/Mainnet reads as appropriate; Walrus Mainnet writes/recalls in controlled test.	No secrets in output; job completion verified; real provider evidence retained safely.
Isolation/security	User A cannot access user B's watchlists, investigations, or memory summaries.	Negative tests fail closed.
Memory loop	Write, await, recall in a separate session, and prove result changes.	At least 10 completed Mainnet memories for submission evidence; repeatable demonstration.
API	Auth, rate limits, error schemas, idempotency, pagination, timeouts.	No uncontrolled upstream proxy; predictable errors.
Frontend	Loading, empty, stale, partial, error, success, keyboard, reduced motion.	No dead controls; accessible focus and labels.
End-to-end	Enter wallet, view evidence, respond to alert, store memory, revisit and recall.	Critical path passes on deployed build.

Test layer	Required coverage	Pass condition
Performance	Representative payloads, slow provider, high latency, large history.	UI remains responsive; requests bounded; graceful degradation.
Dependency/security	Lockfile, vulnerability scan, secret scan, lint, type checks.	No known high-severity issue left unexplained; CI gates documented.

Definition of done

- Code is typed, linted, formatted, reviewed, and covered by appropriate tests.
- No secrets are committed; .env.example contains placeholders only.
- Errors are actionable and never leak credentials or sensitive memory text.
- Critical user journey works against real integrations.
- Documentation includes setup, environment, privacy, known limitations, and demo steps.
- The deployed UI matches the intended light-mode design system and supports reduced motion.

17. Deployment and configuration

Environment configuration

```
APP_ENV=development|staging|production DATABASE_URL=... TATUM_API_KEY=... TATUM_SUI_NETWORK=mainnet
MEMWAL_ACCOUNT_ID=... MEMWAL_PRIVATE_KEY=... MEMWAL_SERVER_URL=https://relayer.memory.walrus.xyz
MEMWAL_NAMESPACE=trace-production AUTH_SECRET=... MODEL_API_KEY=... # optional; server-side only
```

This is a template, not a place to paste credentials into chat or commit them. The Walrus Memory private key and Tatum API key must remain server-side. Use a secret manager in production and separate development/staging/production credentials and namespaces.

Deployment rules

- Pin runtime and dependencies; commit lockfiles and reproducible configuration.
- Use HTTPS in deployed environments and secure cookie/session settings.
- Apply database migrations in a controlled, repeatable manner.
- Expose a readiness check that validates configuration without returning secret values.
- Do not run destructive database migrations automatically on every application boot.
- Set resource limits and request timeouts at the edge and API layer.
- Use a least-privilege database user and restrict production console access.
- Maintain a clear demo mode; label fixture data and never merge it into live wallet state.
- Verify platform build commands, health checks, and environment-variable injection after deployment.

18. Accessibility and performance

Accessibility

- Target WCAG 2.2 AA for contrast, keyboard navigation, focus visibility, labels, and status announcements.
- Do not use colour alone to convey risk or state; pair with text and iconography.
- All charts need accessible summaries or data tables.

- Support reduced motion and avoid interaction that depends solely on hover.
- Use semantic headings, landmarks, form labels, and descriptive error messages.
- Respect browser zoom and reflow on narrow viewports; test at mobile and desktop widths.

Performance and resilience

- Use server-side pagination and bounded history retrieval; never load unlimited wallet history in one request.
- Lazy-load heavy charts and secondary panels; keep the first meaningful wallet summary fast after provider response.
- Use skeletons and progressive rendering only when the data contract supports partial results.
- Debounce user input and prevent duplicate in-flight analyses.
- Set explicit budgets for API timeouts, response payload size, and model context.
- Avoid layout shift when charts, evidence panels, or memory chips appear.
- Measure performance with real devices and network throttling; do not rely on a single developer laptop.

19. Metrics and acceptance criteria

Product metrics

Metric	Definition	Guardrail
Analysis completion rate	Successful wallet analysis responses / valid requests.	Separate provider failure from invalid input.
Evidence coverage	Claims in an explanation linked to evidence / total factual claims.	Target all material transaction claims to be grounded.
Memory write success	Completed memory jobs / attempted writes.	Count only confirmed completed jobs.
Memory usefulness	Sessions where recalled memory materially changes relevant context or explanation.	Measure with reproducible scenario, not vanity counts.
False-alert feedback	Share of alerts users mark expected/incorrect.	Use as feedback, not as definitive fraud ground truth.
Freshness	Age of last successful provider observation at display time.	Show staleness rather than hiding it.
Reliability	Availability and error rates for API and dependencies.	Track each dependency separately.

Release acceptance checklist

- A user can enter a valid Sui address and receive normalized, source-linked data through Tatum.
- The app clearly identifies Mainnet/Testnet and the observation timestamp.
- At least one anomaly rule produces a deterministic, evidence-linked alert in a reproducible test case.
- The explanation distinguishes observed facts, inferences, and uncertainty.
- Walrus Memory successfully writes and recalls in the configured Mainnet environment.
- At least 10 meaningful memories have completed Mainnet write jobs before submission.
- A later session recalls relevant memory and visibly changes the output compared with stateless analysis.
- Memory failures, Tatum failures, and stale data all have honest UI states.
- No Tatum key or Walrus delegate private key appears in the frontend bundle, Git history, logs, screenshots, or public demo.

- Cross-user access tests pass for app-managed data.
- The public repository has a complete README, environment template, architecture diagram, test instructions, and known limitations.
- The demo is reproducible from a clean environment and does not depend on fabricated live data.

20. Demo narrative and judging evidence

Recommended demo sequence

- Open with a real user problem: a wallet has activity, but the raw feed does not explain what is normal or what deserves attention.
- Show TRACE retrieving supported on-chain data and clearly displaying the network, source, and freshness.
- Open a specific alert and trace each claim to a transaction fact.
- Show a user-confirmed memory being written; display completion status rather than an optimistic toast.
- Start a fresh session or clear transient state, then run the same investigation again.
- Show the recalled memory and the exact change it makes to the explanation or priority.
- Show the Memory Centre with provenance and explain what data is public versus private.
- Finish with architecture, tests, public repository, and documented limitations.

Evidence package

Judging criterion	Evidence to prepare
Walrus + Tatum integration	Redacted request evidence, real Tatum response normalized in the app, completed Walrus write job, recall result.
Technical quality	Architecture diagram, test results, API schemas, error handling, CI checks, secure configuration.
Creativity	A before/after memory demo showing changed reasoning and a concrete user outcome.
Presentation	Clear landing page, polished walkthrough, README, technical blog, public demo, limitations page.
Bonus awards	If pursuing AI Builder, document which parts were generated with it. If pursuing x402/MPP, verify exact protocol and Sui settlement requirements before committing to scope.

Do not fabricate transaction activity, provider logs, memory writes, user counts, accuracy metrics, or audit status. If a scenario is seeded for the demo, label it as a controlled test scenario and keep real integration evidence separate.

21. Risks and mitigations

Risk	Impact	Mitigation
Walrus SDK version/API mismatch	Mainnet memory integration fails late.	Verify official SDK and Python support early; write a small authenticated write/recall proof before building features.
Sui endpoint or RPC method changes	Data retrieval breaks or becomes incomplete.	Use current Tatum docs; isolate adapter; add contract tests and clear provider status.
Insufficient wallet history	False or meaningless anomaly scores.	Require minimum samples; label thin history; prefer deterministic explanations.
Memory doesn't affect output	Weak fit for the core hackathon theme.	Make memory retrieval an explicit input to a testable rule and demonstrate stateless vs remembered outputs.

Risk	Impact	Mitigation
Privacy overclaim	Loss of trust and misleading claims.	Document public-chain visibility; classify data; show memory provenance and controls.
Scope explosion	Polish and reliability suffer.	Single-chain, read-only, limited anomaly rules, no custody or transaction execution.
AI hallucination	Misleading risk explanation.	Structured output, evidence references, validation, deterministic fallback, uncertainty language.
External provider outage	Demo unavailable.	Safe cached/stale state, controlled demo fixture clearly labelled, retry and fallback.
Credential exposure	Unauthorized API or memory access.	Server-only secrets, secret scanning, rotation runbook, no keys in chat/screenshots/repo.
Overdesigned UI hides substance	Poor judging despite visual polish.	Every major visual maps to a real data field, action, state, or evidence trail.

22. Repository structure and engineering standards

Suggested repository

```
trace/ apps/ web/ # Next.js, TypeScript, design system, browser tests api/ # FastAPI application
packages/ contracts/ # OpenAPI-generated / shared schemas where practical ui/ # Shared UI components
if monorepo needs them infra/ deployment/ # Deployment config, health checks, runbooks docs/
architecture.md threat-model.md privacy-and-data.md api-contracts.md demo-script.md decisions/ # ADRs:
key architecture decisions tests/ fixtures/ # Sanitized provider responses, never credentials e2e/
.env.example README.md
```

Code standards

- Use explicit domain types and avoid untyped dictionaries across module boundaries.
- Keep provider adapters separate from business logic and UI.
- Use dependency injection for Tatum, memory, model, clock, and repository interfaces to make tests deterministic.
- Keep functions small and name them by domain action; avoid a giant route handler or “AI manager” module.
- Use structured domain errors and central exception mapping.
- Validate every external response and treat chain metadata as untrusted.
- Use database migrations, unique constraints, foreign keys, and explicit indexes.
- Document every architecture decision that affects privacy, memory isolation, provider choice, or deployment.
- Keep commits focused and use CI for formatting, linting, type checking, unit tests, secret scanning, and build verification.
- Do not add Redis, a vector database, microservices, or a queue until a concrete requirement justifies it.

Architecture decisions to record

- Why the chosen Walrus Memory SDK language and adapter are appropriate.
- How user identity maps to Walrus Memory namespace and access controls.
- Which Tatum Sui endpoints are supported and why.
- What information is persisted locally versus only in Walrus Memory.

- How memory deletion, retention, and correction behave.
- How the app differentiates a heuristic alert from a verified security finding.

23. Reference links

Official integration documentation should take precedence over blog posts, copied snippets, or generated code. Confirm the exact endpoint/method and SDK version during implementation.

Tatum Sui RPC documentation — https://docs.tatum.io/reference/rpc-sui
Sui RPC endpoints and supported methods.

Tatum documentation home — https://docs.tatum.io/
API key, RPC and Data API docs.

Tatum Sui gRPC availability — https://docs.tatum.io/changelog/sui-grpc-available-tatum
Current gRPC interface and use cases.

Walrus Memory SDK usage (TypeScript) — https://docs.wal.app/walrus-memory/sdk/usage/memwal
SDK configuration, remember/recall and job completion.

Walrus Memory SDK usage (Python) — https://docs.wal.app/walrus-memory/python-sdk/usage/memwal
Python client usage and configuration.

Walrus Memory architecture — https://docs.wal.app/walrus-memory/fundamentals/architecture/core-components
SDK/relayer responsibilities.

Sui security — https://www.docs.sui.io/develop/sui-architecture/sui-security
Private-key protection and public transaction visibility.

Sui API reference — https://docs.sui.io/references/sui-api
RPC reference and migration guidance.

Stripe — https://stripe.com
Visual hierarchy and product communication reference.

Linear — https://linear.app
Interaction quality and motion reference.

Ramp — https://ramp.com
Financial-product UI reference.

Coinbase — https://www.coinbase.com
Approachable asset and transaction information.

Vercel — https://vercel.com
Developer experience and status-state reference.

Radix UI — https://www.radix-ui.com
Accessible component primitives.

shadcn/ui — https://ui.shadcn.com
Composable component implementation reference.

Framer Motion — https://motion.dev
Animation and reduced-motion implementation reference.

Final product standard

TRACE should be judged by whether its data is real, its memory changes the experience, its explanations are grounded, its security boundaries are honest, and its interface makes complex activity understandable. Visual polish supports that standard; it does not replace it.